

Mattermost Cross Guard CDS Whitepaper

Architecture, deployment, and security analysis for cross-domain message relay

1. Executive Summary

Cross Guard is a Mattermost plugin that bridges isolated Mattermost instances across security domain boundaries using one of four transport providers as the relay layer: NATS (pub/sub), Azure Queue Storage (polling), Azure Blob Storage (batched write-ahead log), or Azure Service Bus (AMQP with PeekLock and dead-letter). It is designed for environments where servers cannot share databases, APIs, or network access, including air-gapped networks, multi-classification deployments, and multi-tenant architectures.

The plugin integrates with Mattermost's shared-channels framework. Outbound, it receives sync events from the framework hooks (`OnSharedChannelsSyncMsg` , `OnSharedChannelsAttachmentSyncMsg` , `OnSharedChannelsProfileImageSyncMsg` , `OnSharedChannelsPing`); inbound, it hands sync events back to the framework via `ReceiveSharedChannelSyncMsg` . Each Mattermost server keeps its own users, teams, channels, and data store. Cross Guard operates purely at the message level, marshalling each sync event into a plugin-owned XML or JSON envelope (the two serializations are field-for-field equivalent) and publishing it onto the configured transport. File attachments and profile images ride a separate file- upload channel with HTTP-style headers carrying the routing metadata.

The wire envelope splits each upstream sync cycle into at most one post per envelope, with non-post content (orphan reactions, memberships, statuses, acknowledgements, mention transforms) riding an optional metadata envelope. This *one-post-per-envelope* contract is a deliberate compliance-content- inspection blast-radius control: rejecting one post does not lose other content as collateral damage. Every outbound envelope is stamped with an `Epoch` (sender process generation) and a per-channel `Sequence` so the receiver can detect duplicates, reorder short gaps, and audit lost ranges. Inbound work is single-active across the cluster: only one node per inbound connection subscribes at a time, gated by a KV TTL lease, so duplicate processing in a multi-node deployment is impossible.

Administrative controls enforce explicit opt-in at both the team and channel level. No messages flow until an administrator deliberately links a connection to a team and then to specific channels within that team. Inbound connections require interactive acceptance via admin prompts before

any data crosses the boundary. In locked-down deployments (`RestrictToSystemAdmins`) the plugin additionally supports two opt-in request-mode flows so team admins (`AllowTeamAdminRequests`) or channel admins (`AllowChannelAdminRequests`) can submit link requests that a higher- privilege admin approves via interactive DM prompts.

Transport security supports TLS 1.2+ across all providers and provider-specific authentication: NATS supports mutual certificate authentication (mTLS), token or credentials authentication, and ACLs for subject-level access control; Azure Queue, Azure Blob, and Azure Service Bus each support either their legacy default credential (storage shared key or SAS connection string) or an Azure AD Service Principal (client-credentials flow) over TLS-always endpoints. Service Principal mode allows credentials to be scoped via Azure RBAC and rotated in Azure AD without touching plugin config; admin guidance for SP setup, the minimum required data- plane roles, and the Federal-grade pattern for keeping the `client_secret` out of plugin config (Mattermost env-var substitution of the connections JSON) are documented in the admin guide.

This whitepaper provides a comprehensive technical analysis of Cross Guard's architecture, message lifecycle, security model, and deployment topologies for audiences evaluating the plugin for cross-domain use.

2. The Cross Domain Problem

What is a Cross Domain Solution?

A Cross Domain Solution (CDS) is a system that enables controlled information transfer between two or more security domains that operate at different classification levels or under different administrative authorities. In government and defense contexts, these domains are often separated by policy, regulation, or physical network segmentation. A CDS must provide a controlled, auditable, and secure pathway for information to cross these boundaries without violating the isolation guarantees of either domain.

Traditional CDS approaches include hardware guards (dedicated appliances that inspect and filter data at the network boundary), data diodes (one-way physical connections that enforce unidirectional flow), and protocol breaks (systems that terminate one protocol and re-originate data in another, preventing direct network connectivity).

These approaches work well for file transfers, email, and structured data exchange. However, they were not designed for real-time collaborative messaging. Modern collaboration tools like

Mattermost operate within a single domain: users on Domain A cannot see or interact with messages on Domain B. There is no native mechanism for bridging conversations across isolated instances.

Why Existing Federation Approaches Fall Short

Several approaches exist for connecting separate Mattermost instances, but each has fundamental limitations for cross-domain use:

Approach	Mechanism	Why It Fails for CDS
Shared Database	Multiple Mattermost instances read/write from the same database	Requires direct network access between domains to the database server. Violates network isolation. The database becomes a single point of compromise spanning both domains.
Direct API Federation	Servers call each other's REST APIs to synchronize data	Requires bidirectional HTTP connectivity between servers. Violates network segmentation policies. Exposes the full Mattermost API surface across the domain boundary.
Email Bridges	Messages forwarded as emails between instances	Loses real-time conversation flow. No edit or delete synchronization. No reaction support. Latency measured in minutes, not milliseconds. Conversations become disjointed threads.
Webhook Relays	Outgoing webhooks trigger incoming webhooks on the other server	Requires direct HTTP connectivity (same problem as API federation). Point-to-point, fragile, no built-in resilience or reconnection. No standardized authentication beyond shared secrets.

What Cross Guard Provides

Cross Guard takes a fundamentally different approach. Rather than synchronizing databases, proxying APIs, or bridging protocols, it operates as a message-level relay with a decoupled pub/sub architecture:

- **Message-level relay, not database sync:** Only individual post events (create, edit, delete, react) cross the boundary. No bulk data transfer, no schema coupling, no shared state.

- **Asynchronous, decoupled architecture:** The two Mattermost servers never communicate directly. A transport intermediary (a NATS message bus, an Azure Storage account hosting a queue or a WAL blob container, or an Azure Service Bus namespace) sits between them. Publishers and subscribers interact only with the transport layer, not with each other.
- **Full server independence:** Each server maintains its own users, teams, channels, database, authentication, and administration. Cross Guard adds relay capability without modifying any of these.
- **Opt-in at every level:** Relay is not automatic. Administrators must explicitly link connections to teams, then to individual channels. Inbound connections require interactive acceptance before any messages flow.
- **Minimal data crossing the boundary:** Only text content, routing metadata, and optionally file attachments transit the transport layer. File transfer is disabled by default and must be explicitly enabled per connection with configurable type filtering and size limits. User metadata, channel properties, post properties, and interactive message elements are intentionally excluded.

3. Transport Layer Options

Cross Guard supports four transport providers: NATS (pub/sub), Azure Queue Storage (polling-based), Azure Blob Storage (batched write-ahead log), and Azure Service Bus (AMQP with PeekLock and dead-letter). Administrators select a provider per connection in the System Console. All four providers carry the same serialized envelope format (JSON or XML) and support the same message types. The choice depends on infrastructure, latency requirements, durability needs, cost profile, and operational preferences.

NATS

What is NATS?

[NATS](#) is a lightweight, high-performance messaging system written in Go. It provides core pub/sub semantics: publishers send messages to named subjects, and subscribers receive messages from those subjects. NATS is designed for cloud-native, distributed systems and is used in production by organizations ranging from startups to government agencies.

For message relay, Cross Guard uses NATS core pub/sub, which is fire-and-forget: messages are delivered to connected subscribers in real time, with no message persistence or replay. For

optional file transfer, Cross Guard uses NATS JetStream Object Store with a hardcoded bucket name of `crossguard-files` (not user-configurable, unlike Azure's `blob_container_name`). This maps directly to Cross Guard's relay model, where messages are relayed as they occur and not queued for later delivery.

Why NATS Fits the CDS Model

1. Decoupled Architecture

Publishers and subscribers never communicate directly. The NATS server mediates all traffic. Neither the publisher nor the subscriber needs to know the other's network address, identity, or even existence. This maps directly to CDS architecture where the transport sits between domains and neither domain has direct access to the other.

2. Subject-Based Routing

Messages are published to named subjects (e.g., `crossguard.relay.a-to-b`). Subscribers filter by subject. This provides natural channel isolation: different relay directions use different subjects, and NATS ACLs can restrict which clients can publish or subscribe to which subjects. No routing configuration is needed beyond subject names.

3. Lightweight Protocol

NATS uses a text-based protocol with minimal overhead. A typical Cross Guard message envelope is a few hundred bytes (JSON or XML). NATS adds negligible framing. The result is sub-millisecond publish latency, which is critical for real-time messaging where users expect near-instant delivery.

4. TLS and Authentication

NATS has native support for TLS 1.2+, token authentication, username/password authentication, and client certificate authentication (mTLS). These map directly to CDS transport security requirements. No additional proxy or TLS terminator is needed.

5. Network Placement Flexibility

A NATS server can be deployed in a DMZ, an enclave, a guard network, or any controlled network segment between domains. The Mattermost servers only need to reach the NATS server; they never need to reach each other. This is the key architectural property that enables CDS deployment.

6. Resilience

NATS clients have built-in reconnection with configurable retry behavior. Cross Guard configures unlimited reconnects with a 2-second wait, so temporary network interruptions (maintenance windows, transient failures) self-heal without manual intervention. Disconnect and reconnect events are logged for operational visibility.

7. ACL Support

NATS server authorization rules can restrict which clients can publish or subscribe to which subjects. In a CDS deployment, this means the NATS administrator can enforce that Server A can only publish to `crossguard.a-to-b` and only subscribe to `crossguard.b-to-a` , and vice versa. This provides transport-level access control independent of the plugin's own authorization.

How Cross Guard Uses NATS

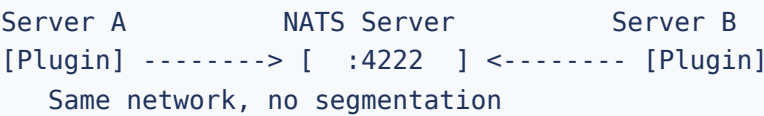
Each Cross Guard connection (inbound or outbound) maps to exactly one NATS subject. Outbound connections publish serialized message envelopes to their configured subject. Inbound connections subscribe to their configured subject and dispatch received messages to type-specific handlers.

The subject prefix `crossguard.` is enforced by configuration validation. All subjects must start with this prefix, which prevents accidental cross-contamination with other NATS traffic if the NATS server is shared (though a dedicated NATS server is recommended for CDS deployments).

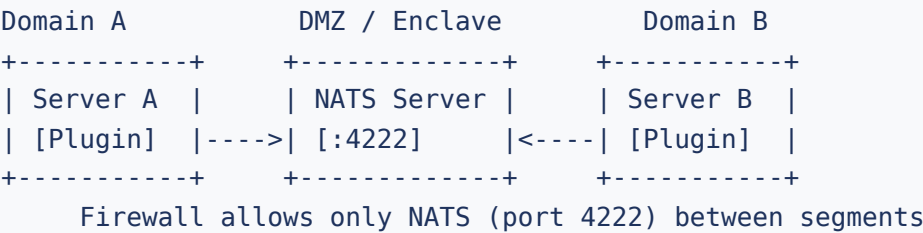
Separate subjects per direction (e.g., `crossguard.a-to-b` and `crossguard.b-to-a`) prevent echo loops at the transport level. Even if the plugin's application-level loop prevention were to fail, messages published on one subject are not received on a different subject.

NATS Deployment Topologies for CDS

Topology 1: Shared Network (Development)



Topology 2: DMZ Placement (Production)



Topology 3: Guard Network (Classified)

Classification A	Guard Appliance	Classification B
+-----+	+-----+	+-----+
Server A	NATS on	Server B
[Plugin]	dedicated guard	[Plugin]
+-----+	hardware/enclave	+-----+
	+-----+	
Dedicated hardware, strict firewall rules, physical separation		

Azure Queue Storage

What is Azure Queue Storage?

[Azure Queue Storage](#) is a fully managed message queuing service provided by Microsoft Azure. It stores messages in queues that can be read by any client with a valid connection string or SAS token. Unlike NATS (which uses real-time pub/sub), Azure Queue Storage uses a polling model where consumers periodically check for new messages.

Why Azure Queue Storage Fits the CDS Model

1. Polling-Based Delivery

The plugin polls the Azure queue on a configurable interval (default 5 seconds for messages, 15 seconds for blob/file transfers). Messages are dequeued in batches of up to 32 with a 5-minute visibility timeout. This polling model is well-suited for environments where persistent connections are difficult to maintain or where store-and-forward semantics are preferred over real-time delivery.

2. At-Least-Once Delivery

Azure Queue Storage provides at-least-once delivery semantics. Messages remain in the queue until explicitly deleted after successful processing. If the consumer crashes before acknowledging, the message becomes visible again after the visibility timeout expires. This is more durable than NATS core pub/sub, which drops messages if no subscriber is connected.

3. Managed Service

Azure Queue Storage is a fully managed service. There is no NATS cluster to deploy, patch, monitor, or scale. The Azure Storage account handles availability, replication, and maintenance. This reduces operational overhead, especially in environments where infrastructure teams prefer managed services over self-hosted components.

4. Connection String Authentication

Authentication uses an Azure Storage connection string, which contains the account name and key. Mattermost does not encrypt plugin settings at rest, so the connection string is stored as

plaintext in `config.json` and the Mattermost configuration database by default; for production, inject the connections JSON via the Mattermost env-var pattern instead (see [Section 11: Secret Storage and Env-Var Injection](#)). No separate TLS certificate management is required because all Azure Storage endpoints use TLS by default.

5. Auto-Provisioning

The plugin automatically creates the queue and blob container (as configured in `queue_name` and `blob_container_name`) on startup if they do not exist. No manual Azure resource provisioning is needed beyond creating the Storage account itself.

How Cross Guard Uses Azure Queue Storage

Each outbound connection enqueues serialized message envelopes (JSON or XML) to the configured Azure queue. File attachments (when enabled) are uploaded to the configured blob container with a key in the format `{postId}/{randomId}`, referenced in the envelope. The inbound side polls the queue, processes each message, and deletes it after successful handling. For files, the inbound side downloads the blob and deletes it after processing.

Azure Blob Storage

What is Azure Blob Storage?

[Azure Blob Storage](#) is a fully managed object store. Cross Guard's Azure Blob transport does not use Azure Queue Storage at all. Instead of enqueueing one message per relay event, the outbound side appends events to an in-memory write-ahead log (WAL) and periodically seals and uploads the WAL as a single batch blob. The inbound side polls the container, takes a short-lived lease on each batch blob, processes its messages, and deletes the blob.

Why Azure Blob Storage Fits the CDS Model

1. Batched Writes Reduce API Cost

One batch blob carries many messages, so total Azure Storage API calls scale with the number of flush intervals rather than with the number of relay events. In high-volume channels this is significantly cheaper than per-message queue operations and easier to reason about for cost forecasting in regulated tenants.

2. Lease-Based Mutual Exclusion

Each batch blob is protected by a short-lived blob lease while an inbound poller processes it. If the poller crashes mid-flight, the lease expires after `blob_lock_max_age_seconds` (default 300)

and another inbound node may break the lease and retry. This gives at-least-once delivery without an external coordinator.

3. Single-Container Footprint

Both batch WAL blobs and deferred file attachment blobs live in the same container. Operators only need to provision one container, which simplifies access policies, audit, and lifecycle rules in CDS environments.

4. Connection String Authentication

Authentication is the same Azure Storage connection string model as the Azure Queue provider. Mattermost does not encrypt plugin settings at rest, so the connection string is stored as plaintext in `config.json` and the Mattermost configuration database by default; inject via env vars for production (see [Section 11: Secret Storage and Env-Var Injection](#)). All Azure Blob traffic is over TLS.

How Cross Guard Uses Azure Blob Storage

Outbound buffers serialized envelopes in memory and every `flush_interval_seconds` (default 60, minimum 5) seals the buffer and uploads it as a time-ordered batch blob. Inbound lists the container on a poll cycle, acquires a lease on each batch blob, dispatches its messages to the inbound handler, then deletes the blob and releases the lease. File attachments are uploaded as deferred file blobs into the same container and reaped after delivery. Expected end-to-end latency is bounded by the configured flush interval.

Azure Service Bus

What is Azure Service Bus?

[Azure Service Bus](#) is a fully managed enterprise message broker speaking AMQP. It provides queue and topic semantics with first-class features that Azure Queue Storage lacks: PeekLock receive with explicit settle, server-side delivery counters, native dead-letter sub-queues, and configurable lock duration on the broker side.

Why Azure Service Bus Fits the CDS Model

1. PeekLock with Explicit Settle

Cross Guard receives messages under PeekLock, then calls Complete on handler success or Abandon on handler error. Service Bus increments `DeliveryCount` on every Abandon and, after the queue's `MaxDeliveryCount` is exceeded, automatically moves the message to the queue's

dead-letter sub-queue. This gives operators poison-message containment without bespoke retry tracking.

2. Pre-Created Queues with ARM-Controlled Tunables

The plugin does *not* auto-create Service Bus queues. Operators provision the queue in the Azure portal, ARM, or Terraform with their chosen `LockDuration` and `MaxDeliveryCount`. Lock duration is an entity property and cannot be set from a client. This puts message-handling SLAs and DLQ policy under the same change-control as other Azure infrastructure.

3. SAS Connection String Authentication

Authentication is via a Service Bus SAS connection string of the form

```
Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<rule>;SharedAccessKey=<key>
```

. Mattermost does not encrypt plugin settings at rest, so the SAS connection string is stored as plaintext in `config.json` and the Mattermost configuration database by default; for production, inject via env vars (see [Section 11: Secret Storage and Env-Var Injection](#)). Service Bus error messages that might leak SAS material are sanitized before logging.

4. Independent File Sidecar

File attachments are transferred via an Azure Blob Storage sidecar container with its own credentials, independent of the Service Bus namespace. Service Bus queues themselves remain message-only and never carry file bytes. This separation lets operators scale and audit each store independently.

How Cross Guard Uses Azure Service Bus

Each outbound connection sends a serialized envelope (JSON or XML) into the configured Service Bus queue via the Azure SDK. The inbound side calls `ReceiveMessages` in batches of up to 32 with a 5-second per-call wait, settles each message with Complete on success or Abandon on error, and logs a WARN when `DeliveryCount > 1` so operators can detect poison-message buildup before DLQ moves happen. The default outbound payload ceiling is 192,000 bytes (Service Bus Standard caps at 256 KiB per message; Premium supports up to 100 MiB). Inbound polls the optional blob sidecar every 15 seconds (configurable) for file attachments.

Transport Comparison

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
Latency	Sub-millisecond (pub/sub)	5-15 seconds (polling)	Bounded by flush interval (default 60s)	Bounded by receive max wait (5s) plus broker latency
Delivery	At-most-once	At-least-once	At-least-once	At-least-once (PeekLock with server-side DeliveryCount)
Message Size	1 MB default (NATS server configurable)	48,000 bytes (before Base64)	Bounded by batch blob size (many messages per blob)	192,000 bytes default; 256 KiB Standard tier ceiling, 100 MiB on Premium
Infrastructure	Self-hosted NATS cluster	Azure managed service	Azure managed service	Azure managed service
Encryption	TLS (opt-in)	TLS always (Azure endpoint)	TLS always (Azure endpoint)	TLS always (AMQP over TLS)
File Transfer	JetStream Object Store (1h TTL)	Blob Storage (deleted after processing)	Same container as WAL (deferred file blobs)	Blob Storage sidecar (independent credentials)
Auto-Provisioning	No (NATS server must exist)	Queue and container auto-created	Container auto-created	No (queue must be pre-created with LockDuration and MaxDeliveryCount)
Best For	Low-latency, high-throughput environments	Azure-native environments, simplicity	High-volume channels where amortized API cost matters	Azure-native environments needing DLQ, redelivery counters, and ARM-controlled SLAs

4. System Architecture

Component Overview



Note: The specific transport mechanism depends on the configured provider. For NATS, messages flow over a NATS subject (e.g., `crossguard.a-to-b`). For Azure Queue Storage, messages flow through an Azure Storage queue. For Azure Blob Storage, messages are appended to an in-memory WAL and flushed periodically as batched blobs into a shared container; the inbound side leases each blob, processes it, and deletes it. For Azure Service Bus,

messages flow through a pre-created Service Bus queue using PeekLock receive with explicit Complete on success or Abandon on error.

Plugin Internal Architecture

The Cross Guard plugin is implemented as a single Go binary that runs inside the Mattermost server process. Its internal architecture consists of the following components:

- **Plugin struct:** The central struct that holds the bot user ID, the KV store reference, the transport provider connections (NATS, Azure Queue, Azure Blob, or Azure Service Bus), the in-process Epoch ID, the per-(connName, channelId) outbound sequence counters, the wait group for goroutine lifecycle tracking, and a context for graceful shutdown.
- **Outbound pool:** A slice of `outboundConn` structs, each holding a `QueueProvider` (NATS connection, Azure Queue client, Azure Blob client with in-memory WAL buffer, or Azure Service Bus sender) and its configuration. Protected by a `sync.RWMutex` to allow concurrent reads (publishing) with exclusive writes (reconnection on config change).
- **Inbound pool:** A slice of `inboundConn` structs, each holding a `QueueProvider` (NATS subscription, Azure queue poller, Azure Blob lease-based reader, or Azure Service Bus PeekLock receiver) plus an elector goroutine that holds the single-active-receiver KV TTL lease and an `inboundSequencer` that dedups envelopes by `(Epoch, ConnName, ChannelId, Sequence)`. The map is protected by a `sync.RWMutex` with the same read / write concurrency model.
- **KV store:** Mattermost's built-in plugin key-value store, used for all persistent state (team and channel linkage, prompt and request state, rewrite indexes, single-active-receiver leases, and inbound sequencer cursors). An optional caching layer wraps the KV store with 15-minute TTL and cluster-aware invalidation.

Plugin Lifecycle

Event	Actions
OnActivate	Creates the bot user, initializes the KV store and caching layer, registers the <code>/crossguard</code> slash command, initializes the HTTP router for the REST API and interactive prompts, creates the shutdown context, generates the per-process Epoch ID, registers the shared-channels remote for every configured connection, connects all configured outbound and inbound providers (NATS, Azure Queue, Azure Blob, or Azure Service Bus, per each connection's

Event	Actions
	<code>provider</code> setting), and starts an elector goroutine per inbound connection so the single-active-receiver lease is acquired immediately. If <code>CROSSGUARD_ENVELOPE_ARCHIVE_DIR</code> is set, the envelope archive is also initialized at this point.
OnConfigurationChange	Validates new configuration, closes existing transport connections, opens new connections with updated settings. Live reconnection with no plugin restart required.
OnDeactivate	Cancels the shutdown context (signals all background goroutines to stop), releases the inbound active-receiver leases, closes inbound subscriptions and file watchers, waits for in-flight goroutines via the WaitGroup, and closes outbound providers.

5. Message Relay Lifecycle (Outbound)

This section traces the complete path of a sync event from the moment Mattermost's shared-channels framework hands it to the plugin on Server A to the moment it is published to the transport layer (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus).

Step 1: Shared-Channels Hook Fires

Mattermost's shared-channels framework drives every cross-server sync. Cross Guard implements four shared-channels hooks in `server/hooks.go` :

- `OnSharedChannelsSyncMsg()` for sync content (posts, edits, deletes, users, reactions, statuses, membership changes, post acknowledgements, mention transforms).
- `OnSharedChannelsAttachmentSyncMsg()` for file attachments.
- `OnSharedChannelsProfileImageSyncMsg()` for user avatars.
- `OnSharedChannelsPing()` for the framework's per-remote health check.

Each hook receives an `mmModel.SyncMsg` (or attachment / image payload) and the originating `RemoteCluster` . The plugin no longer subscribes to the post lifecycle directly; the framework owns post-event detection, filtering of already-synced content, and retry semantics. The hook returns an error to defer cursor advancement, which causes the framework to retry on its own schedule.

Step 2: Outbound Connection Resolution

`connNameForRemote(rc.RemoteId)` maps the framework's remote ID to the corresponding Cross Guard outbound connection name. A sync event arriving from connection A's remote never re-leaves on connection A; this is the per-remote outbound suppression that prevents framework-level relay loops. If no outbound connection is registered for the remote, the hook returns silently.

Step 3: Envelope Fanout (One Post Per Envelope)

`buildOutboundEnvelopes` in `server/transport.go` splits each upstream `*mmModel.SyncMsg` into a fan-out of envelopes, one per Post, plus an optional metadata envelope carrying non-post content (orphan reactions, memberships, statuses, acknowledgements, mention transforms) from the same cycle. Each envelope carries at most one `<Post>` child of `<SyncMsg>`; metadata envelopes have no `<Post>` element at all. This one-post-per-envelope contract is a deliberate compliance- content-inspection blast-radius control: rejecting one post does not lose other content as collateral damage.

On the conversion boundary, upstream `mmModel` types are projected into plugin-owned wire types (`server/wire/`) via per-type `XFromModel` functions. Receiver-overwritten fields (e.g. `Post.ReplyCount` , `Post.Participants`) and source-only fields (`MessageSource` , `PendingPostId`) are dropped at the boundary so they never appear on the wire. `Post.Props` and `User.Props` are projected through the typed `PostProps` and `UserProps` whitelists; any non-whitelisted upstream Props key is dropped.

Step 4: Epoch and Sequence Stamping

`publishToOutboundConn` stamps two ordering fields on every sender- originated envelope before it goes onto the wire:

- `Epoch` : a 26-character Mattermost ID generated once at `OnActivate` and held in process memory for the lifetime of the plugin instance. It identifies the sender process generation; a restart yields a new Epoch so the receiver can reset its per-channel sequence cursor without conflating cycles.
- `Sequence` : a per- (`ConnName` , `ChannelId`) monotonic counter via `KVStore.BumpSequenceCounter` . Lets the receiver detect duplicates and reorder short gaps. Stamped on `sync_msg` envelopes only; absent on `test` envelopes (no channel scope).

Step 5: Serialization

`MarshalEnvelope` serializes each envelope according to the outbound connection's `message_format` setting. XML is the current default and the existing wire format; JSON is the immediate follow-up implementation, and the two formats are field-for-field equivalent (see [the Transport Interface Reference](#) and `schema/crossguard.xsd`). When the `CROSSGUARD_ENVELOPE_ARCHIVE_DIR` environment variable is set, `MarshalEnvelope` also writes a copy of every marshalled envelope to disk so the integration test suite can validate them against the XSD; the archive path is the chokepoint that every outbound publish flows through.

Step 6: Publish to Transport

Each envelope is handed to `QueueProvider.Publish` on the resolved outbound connection. Provider-specific behaviour:

- **NATS** calls `conn.Publish` on the configured subject and returns. The shared-channels framework owns retry: an error from the hook leaves the framework cursor unadvanced, so the framework re-fires the hook.
- **Azure Queue Storage** Base64-encodes the bytes and enqueues with a single Azure SDK call. The plugin enforces a 48,000-byte cap below the 64 KB Azure Queue wire ceiling (Base64 expansion fits).
- **Azure Blob Storage** appends the envelope to an in-memory write-ahead log buffer that is sealed and uploaded as a batch blob every `flush_interval_seconds` (default 60). Many envelopes ride per blob.
- **Azure Service Bus** sends the AMQP message via the Service Bus SDK and enforces a per-connection `max_message_size_bytes` (default 192,000) at the client boundary before sending. SAS-bearing error messages are sanitized before logging.

File attachments and profile images take a parallel path: the corresponding

`OnSharedChannelsAttachmentSyncMsg` / `...ProfileImageSyncMsg` hook downloads bytes from Mattermost and routes them through `QueueProvider.UploadFile` with the metadata-carrying `x-crossguard-*` header set described in [File-Upload Headers](#). The XML envelope path is not used for file payloads.

The hook returns when every envelope has been handed to the provider. There is no plugin-level relay semaphore; concurrency is bounded by the framework's per-remote dispatch and by the provider's own client-side limits.

6. Message Relay Lifecycle (Inbound)

This section traces the complete path of a sync envelope from the moment it arrives via the transport layer (a NATS subject, an Azure Storage queue, a leased Azure Blob batch blob, or an Azure Service Bus queue) to the moment it is handed back to the shared-channels framework on Server B for application to local state.

Step 1: Single-Active Subscription

Across a Mattermost cluster, exactly one node holds the active subscription for a given inbound connection at a time. `server/inbound_election.go` runs an elector goroutine per inbound connection that takes and renews a KV TTL lease (45 s TTL, 15 s renewal, 3x safety ratio). Only the lease-holder calls `QueueProvider.Subscribe`; the other nodes idle and watch for the lease to age out, then race to take it. This design makes duplicate processing in a multi-node deployment impossible (the receiver-side sequencer additionally drops duplicates that survive a handoff).

File events (`QueueProvider.WatchFiles`) follow the same single-active pattern under the same lease, so a node either runs both subscriptions or neither.

Step 2: Bytes In, Envelope Out

Raw bytes arrive at `processInboundMessage` (`server/inbound.go`) from one of the four providers: a NATS subscription callback, an Azure Queue Storage poller (configurable cadence, default 5 s, batches of up to 32), an Azure Blob Storage lease-based reader (one batch blob at a time, scoped by a short-lived blob lease), or an Azure Service Bus PeekLock receiver (batches of up to 32 with a 5 s max wait; Complete on success, Abandon on error). `UnmarshalEnvelope` deserializes the bytes; format is auto-detected by the first non-whitespace byte (`<` for XML, `{` for JSON), and a leading UTF-8 BOM on XML is stripped before detection. The inbound handler is deliberately serial under the single-active lease (no relay semaphore), so the sequencer below can reason about a single ordered input stream per connection.

Step 3: Sequencer Admission

`inboundSequencer.Admit` (`server/inbound_sequencer.go`) keys on (`Epoch`, `ConnName`, `ChannelId`, `Sequence`) and enforces three invariants:

- **Deduplication.** Any envelope whose `Sequence` is at or below the per-channel cursor is dropped. Combined with the single-active lease, this absorbs both at-least-once redeliveries from the

provider and replays caused by a sender restart.

- **Bounded reorder buffer.** An envelope whose Sequence is ahead of the cursor is held in a small in-memory buffer; gap-fill envelopes promote the cursor and release any buffered successors in order.
- **Audited gap-deadline.** A 1-second ticker drains the sequencer of envelopes whose missing predecessors never arrived. The lost ranges are audited via the `InboundSeq*` numeric error codes (see [Error Codes](#)) so operators can surface compliance- relevant gaps without losing the surviving content.

A new `Epoch` resets the per-channel cursor (the sender process generation has changed; the previous cursor is no longer authoritative). A 5- second checkpoint ticker persists cursors to KV under `crossguard-seqcur-{connName}-{channelID}` so leadership handoff resumes from where the previous active node left off.

Step 4: Channel ID Rewriting

Admitted envelopes pass through `rewriteChannelIDs`, which translates source-side channel IDs (carried inside the `SyncMsg`) into local channel IDs using the rewrite index in KV (`crossguard-rwi-{connName}-{remoteTeamName}` for team-level rewrites plus the framework's own channel-pairing state). If the team or channel is not linked to this inbound connection, the handler posts an interactive Accept / Block prompt to `#town-square` (for team-level prompts) or to the target channel (for channel-level prompts) and drops the current envelope; subsequent envelopes follow the receiver-side prompt state until an admin acts.

Step 5: Handoff to the Shared-Channels Framework

With the envelope admitted and the IDs rewritten, the inner `wire.SyncMsg` is projected back into `*mmModel.SyncMsg` via the per-type `ToModel` functions and handed to `p.API.ReceiveSharedChannelSyncMsg`. From this point the framework owns everything: creating, updating, or deleting posts; upserting users; applying reactions and acknowledgements; resolving thread parents; and stamping each received user with a non-empty `RemoteId` that prevents the framework from re-syncing the content back out. The plugin does not maintain its own post-ID mapping or sync-user table.

Step 6: File-Channel Receipt

File payloads (`attachment` and `profile_image`) arrive on the file channel, not the envelope channel: `QueueProvider.WatchFiles` delivers raw bytes plus the metadata-bearing `x-Crossguard-*` headers (see [File-Upload Headers](#)). The handler reads `x-Crossguard-Kind` to discriminate attachments from profile images, runs the same rewrite-then-by-name resolution as the envelope path, and hands off to the framework's attachment / profile-image upsert APIs.

7. Data Boundary Control

Understanding exactly what data crosses the domain boundary is critical for CDS evaluation. This section documents the wire envelope, its payload, and the whitelisting policy that determines which upstream fields are carried and which are dropped at the boundary. The envelope shape is identical across all four transport providers (NATS, Azure Queue Storage, Azure Blob Storage, and Azure Service Bus). JSON and XML are field-for-field equivalent serializations of the same semantic envelope; the XSD at `schema/crossguard.xsd` is the normative structural contract.

Two Delivery Paths

The four envelope types are routed through two distinct delivery paths:

- **XML / JSON envelope path** (`QueueProvider.Publish` / `Subscribe`): carries `sync_msg` and `test` envelopes. The `<CrossGuardEnvelope>` wraps a `<SyncMsg>` (for `sync_msg`) or a `<TestID>` (for `test`). Each `sync_msg` envelope carries at most one Post.
- **File-upload path** (`QueueProvider.UploadFile` / `WatchFiles`): carries `attachment` and `profile_image` payloads. The raw file bytes ride the transport's native file-upload channel; routing metadata travels as HTTP-style `x-Crossguard-*` headers on the file blob (see [File-Upload Headers](#)). The XML / JSON envelope is not used for file payloads.

Envelope Schema

Each `<CrossGuardEnvelope>` carries the following envelope-level fields. Compliance content inspection can route and audit envelopes by inspecting these alone, without parsing the inner `SyncMsg` .

Field	Content	User-Controlled?	Used For
<code>version</code> (attribute)	Integer, currently always <code>1</code>	No	Wire-format versioning
<code>type</code> (attribute)	One of <code>sync_msg</code> , <code>test</code> , <code>attachment</code> , <code>profile_image</code>	No	Envelope dispatch (only <code>sync_msg</code> and <code>test</code> appear in the XML envelope itself)
<code>ConnName</code>	Outbound connection name (URL-safe slug, max 96 chars)	No (admin-set)	Routing, audit, per-channel sequence scoping
<code>Timestamp</code>	RFC 3339 UTC timestamp set at marshal time	No	Audit / logging
<code>Epoch</code>	Sender process generation (26-char Mattermost ID, generated at plugin activation)	No	Sequence cursor reset on sender restart; replay detection
<code>Sequence</code>	Per- (<code>ConnName</code> , <code>ChannelId</code>) monotonic counter (<code>sync_msg</code> only)	No	Duplicate detection, bounded reorder
<code>TeamName</code>	Source team name (URL-safe slug, max 96 chars)	Indirectly (admin-set)	Routing via name resolution and the rewrite index
<code>ChannelName</code>	Source channel name (URL-safe slug, max 96 chars)	Indirectly (admin-set)	Routing via name resolution
<code>SyncMsg</code>	Plugin-owned <code>wire.SyncMsg</code> (see below); present when <code>type</code> is <code>sync_msg</code>	Contains user-controlled fields (see <code>SyncMsg Contents</code> below)	Post / user / reaction / status / membership / acknowledgement upserts
<code>TestID</code>	26-char ID echoed back by the receiver; present when <code>type</code> is <code>test</code>	No	Connectivity verification

Wire Format Encoding: JSON and XML

Each outbound connection chooses its on-the-wire encoding via the `message_format` setting (`"xml"` or `"json"`). JSON field names exactly mirror the XML element / attribute names, so the two serializations are field-for-field equivalent and the same semantic envelope schema applies. XML is rooted at `<CrossGuardEnvelope>` (no XML namespace) and validated against `schema/crossguard.xsd` by construction. Inbound connections do not require format configuration: the parser inspects the first non-whitespace byte (`<` means XML, `{` means JSON) and strips a leading UTF-8 BOM if an upstream device prepended one.

The repository ships 22 schema-valid XML sample payloads under `schema/examples/` . Each file is byte-for-byte identical to the bytes `MarshalEnvelope` produces. `TestExampleFiles` in `server/examples_roundtrip_test.go` enforces that contract and verifies that `UnmarshalEnvelope` followed by `MarshalEnvelope` is a fixed point. Operators evaluating the protocol for CDS approval can point at these example files as the canonical wire shape and at the test as evidence that the plugin will not silently drift away from them. Field-level references for both serializations live in [transport-interface.html](https://github.com/mattermost/mattermost/blob/master/server/transport-interface.html).

SyncMsg Contents

The `<SyncMsg>` payload is a plugin-owned wire-pruned subset of Mattermost's upstream `mmModel.SyncMsg` . The wire types live under `server/wire/` ; each is a pruned version of the corresponding upstream type with source-only fields stripped and receiver-overwritten fields dropped at the source. The compliance-relevant constraints:

- **One Post per envelope.** Sender-side fanout (`buildOutboundEnvelopes`) splits each upstream sync cycle into one envelope per Post, plus an optional metadata envelope (no `<Post>`) carrying non-post content from the same cycle. This bounds the compliance blast radius: rejecting one post envelope cannot lose other content.
- **Whitelisted typed Props.** `Post.Props` and `User.Props` are not pass-through maps. They are the typed `PostProps` and `UserProps` structs with a fixed set of keys (15 keys on `PostProps` , 4 on `UserProps`). Any non-whitelisted upstream key is dropped at the boundary, regardless of which plugin set it. The full key list lives in [PostProps](#) and [UserProps](#).
- **Dropped at source.** Upstream fields the receiver overwrites (`Post.ReplyCount` , `Post.LastReplyAt` , `Post.Participants` , `Post.Metadata` , `User.AuthService` , `User.NotifyProps` , etc.) or that the framework computes locally (`Post.Filenames`) never appear on the wire.

- **Username scrubbing.** Colon-tagged remote usernames (`alice.high`) are resolved to their pre-tag form before being placed in `Username` . The colon-bearing form, if needed downstream, is recovered from `UserProps.RemoteUsername` on the receiver. This keeps source-side remote topology off the wire.
- **Status pruning.** Source-local `ActiveChannel` on presence updates is dropped at the boundary.

The complete per-type field list, with all whitelisted keys and XSD-enforced length caps, is in [the SyncMsg Payload reference](#).

File-Upload Path: Headers, Not Envelopes

File attachments and profile images do not ride the XML / JSON envelope. They flow through `QueueProvider.UploadFile` as raw bytes with a small set of HTTP-style headers. Each transport maps the header set onto its native metadata mechanism (NATS object store headers, Azure Blob metadata, Azure Service Bus blob sidecar metadata). The header set is documented in [File-Upload Headers](#); the key facts:

- File transfer is **opt-in per connection** via the `file_transfer_enabled` setting (default false).
- File extension allow / deny filters can be configured per connection via `file_filter_mode` and `file_filter_types` ; the policy applies on both the outbound and inbound sides.
- File size is bounded by Mattermost's `FileSettings.MaxFileSize` (default 100 MB).
- File attachments include `x-File-Info` , a Base64-encoded JSON `mmModel.FileInfo` . Base64 wraps the JSON so it survives provider metadata-encoding rules; the receiver decodes it and trusts only the framework to apply the attachment to the local post.
- Object keys follow stable patterns (`attachment/{postID}/{fileID}` for attachments, `profile_image/{userID}` for profile images), and the receiver cross-checks the key prefix against the `x-Crossguard-Kind` header so a mistyped upload cannot escape its lane.

8. User Identity Across Domains

User identity is handled by Mattermost's shared-channels framework, not by Cross Guard. Every `User` record that arrives in an inbound `SyncMsg` is stamped with a non-empty `RemoteId` identifying the source remote. The framework upserts these users as *remote users*: local accounts that are clearly distinguishable from native users by their `RemoteId` , by the framework-managed `RemoteUsername` in the typed `UserProps` , and by the webapp's "Relayed from" indicator on the user

popover. There is no username-collision impersonation risk: the framework will not silently merge a remote user into an existing local account that has a different `RemoteId` (including the empty `RemoteId` that identifies native users).

On the outbound side, the plugin normalizes usernames before they reach the wire so colon-tagged remote forms (e.g. `alice.high`) are resolved to the pre-tag form (`alice`) and stored in the wire `Username` field. The colon-bearing form, if needed downstream, is carried in `UserProps.RemoteUsername` for the receiving framework to recover. This keeps source-side remote topology off the wire while preserving the receiver's ability to display the canonical attribution.

User identity at the boundary is therefore the responsibility of the framework plus the wire-types whitelist:

- **What crosses the boundary:** the whitelisted `wire.User` fields (Id, identity strings, names, position, roles, locale, optional Timezone, and the four-key `UserProps` whitelist). Credentials, `AuthService`, `NotifyProps`, MFA state, password-update timestamps, and similar source-local data are dropped at the boundary; see Section 7.
- **What stops at the boundary:** any upstream Props key not in the `UserProps` whitelist. Plugin-specific or compliance-marking Props that other plugins might set on a user are dropped at the wire types conversion, regardless of which plugin produced them.
- **What the framework guarantees:** received users land in the local user table with a stable, non-empty `RemoteId`; further sync events from the same source resolve to the same local user record, even after renames on the source side, because the framework keys on (Remoteld, source Id) rather than on username.

The webapp's `CrossguardUserPopover` component reads `UserProps.RemoteUsername` and `RemoteId` to display a "Relayed from: alice (via high-side)" attribution on synced users, so the operator audience can always tell which conversation came from which domain.

9. Connection Management and Administrative Workflows

Connection Configuration (System Console)

Transport connections are configured in the Mattermost System Console under **Plugins > Cross Guard**. Inbound and outbound connections are configured as JSON arrays. Each connection specifies a provider type (NATS, Azure Queue Storage, Azure Blob Storage, or Azure Service Bus) along with provider-specific settings:

- **name:** Unique identifier, lowercase alphanumeric with hyphens (e.g., `relay-to-server-b`)
- **provider:** Transport provider type (`"nats"` , `"azure-queue"` , `"azure-blob"` , or `"azure-servicebus"`). Provider-specific fields are nested under the corresponding key.

NATS Provider Fields (nested under `"nats"`)

- **address:** NATS server address (e.g., `nats://nats:4222`)
- **subject:** NATS subject to publish/subscribe (must start with `crossguard.`)
- **TLS settings:** `tls_enabled` , `client_cert` , `client_key` , `ca_cert`
- **Auth settings:** `auth_type` (`none` , `token` , or `credentials`), plus type-specific fields

Azure Queue Provider Fields (nested under `"azure_queue"`)

- **queue_service_url:** Azure Queue Storage service URL (e.g., `https://<account>.queue.core.windows.net`)
- **blob_service_url:** Azure Blob Storage service URL, required when `file_transfer_enabled` is true
- **account_name / account_key:** Shared-key credentials. Treated as secrets; see [Section 11: Secret Storage and Env-Var Injection](#) for the recommended env-var pattern that keeps these out of `config.json` and the Mattermost configuration database.
- **queue_name:** Name of the Azure Storage queue (user-configurable, created automatically if it does not exist)
- **blob_container_name:** Name of the Azure Blob container for file transfers (user-configurable, created automatically if it does not exist)

Azure Blob Provider Fields (nested under `"azure_blob"`)

- **service_url:** Azure Blob Storage service URL
- **account_name / account_key:** Shared-key credentials. Treated as secrets; see [Section 11: Secret Storage and Env-Var Injection](#).
- **blob_container_name:** Container that holds both batch WAL blobs and deferred file blobs (auto-created if missing)
- **flush_interval_seconds:** Interval at which the outbound WAL buffer is sealed and uploaded as a batch blob. Default 60 seconds, minimum 5.
- **blob_lock_max_age_seconds:** Maximum lease age before a stall is broken by another inbound poller. Default 300 seconds, minimum 30.

Azure Service Bus Provider Fields (nested under `"azure_servicebus"`)

- **connection_string:** SAS connection string for the Service Bus namespace. Treated as a secret; see [Section 11: Secret Storage and Env-Var Injection](#).
- **queue_name:** Pre-created Service Bus queue name or hierarchical path. Queues are NOT auto-created; operators must provision them in the Azure portal or via ARM/Terraform with the desired `LockDuration` and `MaxDeliveryCount` .
- **blob_service_url:** Azure Blob Storage service URL for the file attachment sidecar, required when `file_transfer_enabled` is true
- **blob_account_name / blob_account_key:** Shared-key credentials for the blob sidecar storage account (independent of the Service Bus namespace). Treated as secrets; see [Section 11: Secret Storage and Env-Var Injection](#).
- **blob_container_name:** Blob sidecar container for file attachments (auto-created if missing), required when `file_transfer_enabled` is true
- **max_message_size_bytes:** Maximum outbound payload size. Default 192000; maximum 104857600 (100 MiB, Service Bus Premium tier).
- **blob_poll_interval_seconds:** Inbound poll cadence for the blob sidecar. Default 15, minimum 1.

Configuration changes trigger live reconnection. The plugin closes existing transport connections and opens new ones with the updated settings. No plugin restart is required.

Team Initialization Workflow

- 1 **Admin runs** `/crossguard init-team [connection-name]` in the team to be linked. If only one connection exists, it is auto-selected. If multiple exist, an interactive dialog with a dropdown is presented.
- 2 **Connection linked to team:** The connection is stored in the KV store using an atomic compare-and-set (CAS) operation to prevent race conditions.
- 3 **Announcement posted:** The bot posts in `#town-square` : "Cross Guard connection outbound:relay-to-b linked to this team by @admin."
- 4 **Global tracking updated:** The team ID is added to the global initialized teams list for status reporting.

Channel Initialization Workflow

- 1 **Admin runs** `/crossguard init-channel [connection-name]` in the channel. The slash command requires the team to be initialized first via `/crossguard init-team`. The REST API auto-links the team if needed.
- 2 **Connection linked to channel:** Stored in KV store via CAS operation.
- 3 **Visual indicator added:** Channel header is updated with a link emoji prefix for visual indication of relay status.
- 4 **WebSocket event published:** The webapp receives a notification to update channel indicators in real time.
- 5 **Announcement posted:** Bot posts a confirmation in the channel.

Inbound Acceptance Workflow

When the first inbound message arrives for an unlinked team or channel, the plugin does not silently create the link. Instead, it presents an interactive prompt to administrators:

1. Bot posts an interactive message with Accept and Block buttons to `#town-square` (for team-level) or the target channel (for channel-level)
2. Only one prompt is created per team-connection pair (idempotent via KV store check)
3. A Team Admin or System Admin clicks Accept: the connection is auto-linked, and the prompt is replaced with a success message
4. If Block is clicked: the prompt state is set to "blocked" and no messages flow until `/crossguard reset-prompt` clears it
5. The same flow repeats at the channel level once the team is accepted

Request-Mode Workflows (Locked-Down Deployments)

In deployments where `RestrictToSystemAdmins` is enabled, team and channel admins lose direct access to `init-team` and `init-channel`. Two opt-in settings restore a request-and-approve flow so non-sysadmin admins can still drive cross-domain setup without granting them unsupervised linking authority.

Team Admin Requests (`AllowTeamAdminRequests`)

1. A Team Admin runs `/crossguard init-team [connection]` . The plugin detects request mode and, instead of linking, persists a request entry under `crossguard-connreq-{teamID}-{connName}` via CAS.
2. The plugin sends a direct-message announcement to every System Admin with an interactive prompt carrying Approve and Deny buttons. The prompt summarizes the requesting team, the proposed connection, and the requester's user info.
3. Approve triggers `POST /api/v1/request/approve` : the team is linked, the standard init announcement is posted in the requesting team, and the request entry is removed.
4. Deny triggers `POST /api/v1/request/deny` , which opens an interactive dialog so the System Admin can record an optional reason. The dialog submit is handled by `POST /api/v1/request/deny-submit` , which removes the request and DMs the requester with the reason.
5. `teardown-team` remains available to Team Admins in request mode (no approval needed; unlinking does not enlarge attack surface).

Channel Admin Requests (`AllowChannelAdminRequests`)

1. A Channel Admin runs `/crossguard init-channel [connection]` . The plugin persists a channel-level request under `crossguard-chanconnreq-{channelID}-{connName}` via CAS, mirroring the team-level flow.
2. Approval and denial go to **Team Admins** rather than System Admins, via `POST /api/v1/channel-request/approve` , `.../deny` , and `.../deny-submit` . Team Admins are already trusted to link channels in non-restricted deployments, so this preserves the existing trust boundary.
3. `teardown-channel` remains available to Channel Admins in request mode.

Both request-mode flows are **opt-in** and require `RestrictToSystemAdmins` to be on. With `RestrictToSystemAdmins` off, the legacy direct-link flow applies and the request endpoints are inert.

Team Name Rewriting

For environments where the remote and local team names differ (common in multi-domain deployments where each domain has its own naming conventions):

- `/crossguard rewrite-team [connection] [remote-team-name]` creates a mapping from the remote team name to the local team where the command is run
- Stored as a reverse index in KV: `crossguard-rwi-{connName}:: {remoteTeamName}` maps to the local team ID

- Conflict detection: the same remote team name cannot map to two different local teams (returns HTTP 409)
- Audit trail: bot posts in `#town-square` showing who set or cleared the rewrite

Teardown Workflow

- `/crossguard teardown-channel` and `/crossguard teardown-team` unlink connections
- Removes the link emoji from the channel header
- Posts a farewell announcement
- Does not cascade: `teardown-team` does not automatically teardown channels within the team

10. Loop Prevention Architecture

The Loop Problem

Relay loops are the most critical failure mode for any message relay system. Without prevention, content received from a remote could be re-emitted back to that remote (or to other remotes), multiplying messages indefinitely.

Mechanism 1: Framework Cursors and RemoteId

Mattermost's shared-channels framework is the primary loop-prevention mechanism. Every `User` upserted from an inbound `SyncMsg` is stamped with a non-empty `RemoteId` identifying the source remote, and every Post / Reaction / Acknowledgement is similarly attributed. The framework's sync cursors key on `(RemoteId, source content ID)` rather than on local timestamps. Content marked as having come from a remote is not eligible to be re-synced out by the framework, because the framework recognises it as already present at the source. This is the framework-intrinsic guarantee Cross Guard inherits by virtue of running on top of the shared-channels API.

Mechanism 2: Per-Remote Outbound Suppression

On top of the framework's intrinsic guarantee, the plugin's outbound hooks resolve the destination connection via `connNameForRemote(rc.RemoteId)` in `server/hooks.go`. If the framework asks the plugin to sync content *to* the same remote it came *from*, the plugin returns silently instead of

publishing. A sync envelope that arrived on connection A never re-leaves on connection A, even if the framework cursor logic ever produced a candidate outbound event for it.

Mechanism 3: Connection Linkage Validation

Envelopes are only published to outbound connections explicitly linked to the originating team. Inbound envelopes are only processed if the target team and channel are linked to the receiving inbound connection. Both directions enforce opt-in: an unlinked context posts an Accept / Block prompt rather than applying content, and a misconfigured connection cannot leak content to a team that has not consented to relay.

Mechanism 4: Inbound Deduplication (Sequencer)

The inbound sequencer (`server/inbound_sequencer.go`) keys on (Epoch, ConnName, ChannelId, Sequence) and rejects any envelope it has already admitted. Combined with the single-active-receiver KV TTL lease (`server/inbound_election.go`), this catches duplicates that a misbehaving or at-least-once provider redelivers, replays caused by a sender restart, and any handoff window during leader change in a clustered deployment. Lost ranges are audited via the `InboundSeq*` numeric error codes so operators can detect compliance-relevant gaps without losing the surviving content.

Defense in Depth

Layered, independent mechanisms

The four mechanisms operate independently. The framework guarantees no content-marked-as-remote ever re-sources to the framework's own outbound queue. The plugin's per-remote suppression catches anything the framework's guarantee might miss. Connection linkage validation enforces explicit opt-in. The sequencer absorbs duplicate delivery from the transport. No single failure mode can cause a loop without simultaneously defeating all four layers.

11. Transport Security Configuration

NATS Transport Security

TLS Configuration

- Minimum TLS 1.2 enforced (hardcoded `tls.VersionTLS12` in the NATS connection options)

- Connection fields: `tls_enabled` (boolean), `client_cert` (file path), `client_key` (file path), `ca_cert` (file path)
- Client certificate and key must be provided together (validation enforced at configuration time)
- CA certificate enables server certificate verification (the plugin verifies the NATS server's identity)

Authentication Options

Method	Configuration	Description	Suitable For
<code>none</code>	No additional fields	No NATS authentication. Anyone who can reach the NATS server can publish and subscribe.	Development and testing only
<code>token</code>	<code>token</code> field	Bearer token sent on NATS connection. Shared secret between the plugin and the NATS server.	Production environments with simple auth requirements
<code>credentials</code>	<code>username</code> + <code>password</code> fields	Username and password sent on NATS connection. Allows per-client identity on the NATS server.	Production and CDS environments requiring per-client ACLs

Connection Resilience

- Persistent connections with unlimited reconnects (`MaxReconnects = -1`)
- 2-second reconnect wait between attempts
- Disconnect events logged at WARN level, reconnect events at INFO level
- Context-aware cancellation: reconnection attempts are abandoned during graceful plugin shutdown

Outbound Publish Reliability

- 3 retry attempts with exponential backoff (500ms, 1s, 2s)
- Each publish followed by a flush call to confirm delivery to the NATS server
- Messages are dropped (not queued) after retry exhaustion
- Context cancellation is respected during backoff waits

Configuration Validation

- **Connection names:** lowercase alphanumeric with hyphens, unique across all connections (both inbound and outbound)
- **Subject:** must start with `crossguard.` prefix (prevents accidental subscription to unrelated NATS traffic)
- **Auth type:** must be `none` , `token` , or `credentials`
- **Conditional fields:** token required for token auth, username and password required for credentials auth

Deployment Sensitivity Guide

Level	TLS	Auth	mTLS	Dedicated NATS	ACLs
Development	Optional	Optional	No	No	No
Production	Required	Token or credentials	Recommended	Recommended	Recommended
CDS (classified)	Required	Credentials	Required	Required	Required

Azure Transport Security

All three Azure providers support two authentication modes, selected per connection via `auth_mode` :

Provider	Legacy mode (<code>auth_mode</code> default)	Service Principal mode (<code>auth_mode</code> : "service-principal")
Azure Queue Storage	<code>shared-key</code> : <code>account_name</code> + <code>account_key</code>	<code>tenant_id</code> + <code>client_id</code> + <code>client_secret</code> ; <code>account_key</code> must be empty
Azure Blob Storage	<code>shared-key</code> : <code>account_name</code> + <code>account_key</code>	<code>tenant_id</code> + <code>client_id</code> + <code>client_secret</code> ; <code>account_key</code> must be empty
Azure Service Bus	<code>connection-string</code> : SAS connection string	<code>tenant_id</code> + <code>client_id</code> + <code>client_secret</code> +

Provider	Legacy mode (<code>auth_mode</code> default)	Service Principal mode (<code>auth_mode: "service-principal"</code>)
	(<code>Endpoint=sb://...;SharedAccessKeyName=...;SharedAccessKey=...</code>)	<code>service_bus_namespace</code> ; <code>connection_string</code> must be empty

Why Service Principal mode matters for CDS. Shared keys and SAS connection strings cannot be scoped via Azure RBAC and cannot be rotated through Azure AD; rotation requires regenerating the storage key (which rotates every consumer) or revoking the SAS rule on the Service Bus namespace. Service Principal mode authenticates through the Azure AD client-credentials flow and accepts a `azure_cloud` setting (`public` , `usgov` , `china`) so the Azure AD authority matches the deployment cloud. The credential can be scoped via Azure RBAC role assignments on the specific resource (*Storage Queue Data Contributor* on the queue, *Storage Blob Data Contributor* on the container, *Azure Service Bus Data Sender/Receiver* on the queue) and rotated in Azure AD without touching plugin config. Service Principal mode is the recommended posture for Federal and any production deployment.

Auto-provisioning policy. In legacy / shared-key mode the Azure Queue and Azure Blob providers auto-create their queue and container on startup. In Service Principal mode auto-provisioning is **skipped**: operators must pre-provision the queue (and, for file transfer, the container) so the Service Principal can run with data-plane-only RBAC. The Service Bus provider never auto-creates queues in either mode; operators must pre-create the queue with the desired `LockDuration` and `MaxDeliveryCount` .

- **Credential handling.** Shared keys, SAS connection strings, and Service Principal `client_secret` values are treated as secrets. The plugin sanitizes `SharedAccessKey=` , `SharedAccessSignature=` , and `sig=` material out of any Service Bus error messages it logs (`sanitizeServiceBusError()`). See [Secret Storage and Env-Var Injection](#) below for how to keep credentials out of `config.json` .
- **Independent sidecar credentials (Service Bus).** When the Azure Service Bus provider has `file_transfer_enabled` and the parent uses connection-string mode, the blob sidecar takes its own `blob_account_name` / `blob_account_key` credentials separate from the Service Bus namespace. The two stores can live in different Azure subscriptions and have independent access policies. In Service Principal mode the sidecar inherits the parent Service Principal credential (mixed-mode sidecars are not supported), so the SP must hold blob RBAC on the sidecar container in addition to the Service Bus queue.

- **Network security.** Azure Storage and Azure Service Bus both support Virtual Network service endpoints and private endpoints to restrict access to specific networks. Service Principal mode is compatible with both.
- **Encryption in transit and at the provider.** All Azure Storage traffic uses TLS, and Azure Service Bus uses AMQP over TLS. Data at rest is encrypted by Azure Storage Service Encryption (SSE) and by Service Bus's managed encryption at rest. (See the next subsection for credential storage on the Mattermost side, which is a separate concern.)
- **Access control posture.** For shared-key and connection-string modes, prefer scoped Service Bus SAS rules over the namespace-wide `RootManageSharedAccessKey` so an exposed credential grants only the minimum needed rights. For Service Principal mode, scope role assignments to the specific resource (queue, container) rather than the storage account or namespace.
- **SP credential audit.** On each successful Service Principal credential construction, the plugin emits an INFO-level audit log (error code `12010` , `AzureSPAuditConstructed`) recording the tenant ID, client ID, Azure cloud, and the first 12 hex characters of SHA-256 of the secret. This gives operators an audit trail for SP rotations without ever logging the secret value.

Secret Storage and Env-Var Injection

Mattermost does not encrypt plugin settings at rest. By default, the connections JSON (`OutboundConnections` / `InboundConnections`), including any embedded NATS tokens or passwords, Azure Storage shared keys, or Service Bus SAS connection strings, is stored as plaintext in `config.json` on disk and in the Mattermost configuration database. Any operator with disk, backup, or DB access can read these credentials.

For CDS and any production deployment, inject the connections JSON via Mattermost's environment-variable substitution instead of saving it through the System Console. Mattermost overrides plugin settings at config-load time from environment variables of the form:

```
MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS=<json-array>
MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_INBOUNDCONNECTIONS=<json-array>
```

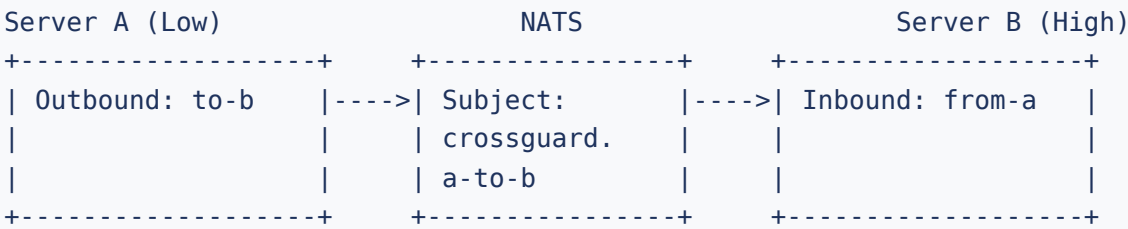
Source these environment variables from a secret store appropriate for your platform: Kubernetes secrets, systemd `LoadCredential=` , HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, or equivalent. The credential then lives only in the Mattermost process environment, not in `config.json` on disk and not in the Mattermost configuration database. Disk and DB backups no longer carry the secret.

Residual exposure with env-var injection: the credential is still readable by anyone with read access to the Mattermost process environment (root on the host, container exec, `/proc/<pid>/environ` , container orchestrator logs that capture the env). System Admins can also still see the in-memory plugin settings through the System Console UI. Combine env-var injection with key rotation and scoped SAS rules so a compromised credential can be rotated quickly and grants only the minimum needed rights.

12. Deployment Topologies

Cross Guard supports multiple deployment topologies depending on the number of servers, the direction of message flow, the transport provider, and the network architecture between domains. Topology diagrams for each transport follow: NATS topologies first, then one diagram per Azure transport provider (Queue Storage, Blob Storage, Service Bus).

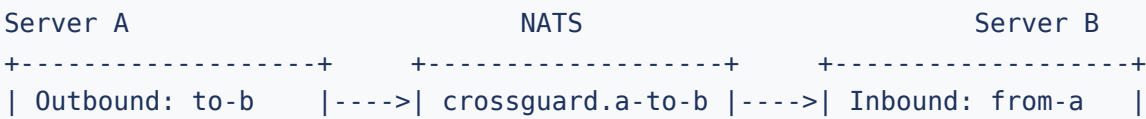
Topology 1: Unidirectional Relay (Low-to-High)



Messages flow one direction only: Low --> High

- Server A has one outbound connection; Server B has one inbound connection
- Same NATS subject configured on both sides
- Messages flow from the low-side to the high-side only
- Use case: low-side collaboration visible on high-side for monitoring or awareness

Topology 2: Bidirectional Relay



```

| Inbound: from-b    |<----| crossguard.b-to-a |<----| Outbound: to-a    |
+-----+           +-----+           +-----+

```

Two subjects, two directions: full two-way conversation flow

- Each server has one outbound and one inbound connection
- Different NATS subjects per direction prevents echo at the transport level
- Full two-way conversation flow between domains
- Use case: collaborative communication between domains

Topology 3: Hub-and-Spoke (Multi-Server Broadcast)

```

                                NATS
                                +-----+
Server A (Hub)                 | Subject: | Server B
+-----+                     | crossguard. | +-----+
| Outbound: to-all |-->| hub-broadcast |-->| Inbound: from-hub |
+-----+                     |           | +-----+
                                |           |
                                |           | Server C
                                |           | +-----+
                                |           | |-->| Inbound: from-hub |
                                |           | +-----+
                                |           |
                                |           | Server D
                                |           | +-----+
                                |           | |-->| Inbound: from-hub |
                                +-----+ +-----+

```

One publisher, multiple subscribers on the same subject

- Server A publishes; Servers B, C, D subscribe to the same subject
- NATS delivers to all subscribers automatically
- Use case: broadcast from one domain to many receiving domains

Topology 4: Loopback (Same-Server Relay)

Server A

```
+-----+
| Outbound: loopback --> NATS --> Inbound: loopback |
|                                                     |
| Team "source-team"                                |
|   Channel: #ops-updates                            |
|   |                                                  |
|   Team Name Rewrite:                              |
|   "source-team" --> "dest-team" (local)            |
|   |                                                  |
| Team "dest-team"                                  |
|   Channel: #ops-updates                            |
+-----+
```

Both connections on the same server, team name rewrite routes between teams

- Both outbound and inbound connections on the same server, same NATS subject
- Requires team name rewrite to route messages from one team to another
- Use case: internal relay between isolated project teams on the same server

Topology 5: Multi-Hop Relay

Server A		NATS 1		Server B		NATS 2		Server C
+-----+		+-----+		+-----+		+-----+		+-----+
Outbound	---->	cg.a-b	---->	Inbound from-a				
		+-----+		Outbound to-c	---->	cg.b-c	---->	Inbound
+-----+				+-----+		+-----+		+-----+

Server B acts as a relay point. Each hop is independent.

- Server B receives from A and re-relays to C
- Each hop is an independent Cross Guard configuration with its own connections
- Messages on Server B are full local posts (editable, deletable, visible to Server B users)
- Use case: cascading classification levels (e.g., Unclassified to Secret to Top Secret)

Azure Queue Storage Topology

In Azure-native deployments, both Mattermost instances connect to a shared Azure Storage account. The outbound server enqueues messages and uploads file blobs. The inbound server polls the queue and blob container. No additional infrastructure (NATS cluster, JetStream) is required.

Server A (Low Side)	Azure Storage Account	Server B (High Side)
+-----+	+-----+	+-----+
	Queue: (configured)	
Enqueue message		Poll queue (5s)
	Blob: (configured)	
Upload blob		Poll blobs (15s)
+-----+	+-----+	+-----+

Azure Blob Storage Topology

The Azure Blob transport replaces the per-message queue with a batched write-ahead log in a single blob container. Outbound buffers events in memory and every `flush_interval_seconds` seals the buffer and uploads it as a time-ordered batch blob. Inbound takes a short-lived blob lease per batch, processes the messages, then deletes the blob. File attachments live in the same container as deferred file blobs. There is no Azure Queue Storage dependency.

Server A (Low Side)	Azure Storage Account	Server B (High Side)
+-----+	+-----+	+-----+
	Container: shared	
Append to WAL	- batch blobs (WAL)	List + lease batch
Flush every 60s	- deferred file blobs	blob, deliver, then
Upload file blob		delete blob
(deferred)	(lease = mutual	Stale lease (>300s)
	exclusion per blob)	may be broken
+-----+	+-----+	+-----+

Azure Service Bus Topology

The Azure Service Bus transport uses a pre-created Service Bus queue for messages and an optional Azure Blob Storage sidecar container for file attachments. Authentication is via a SAS connection string for the Service Bus namespace and independent storage account credentials for the blob sidecar. Inbound uses PeekLock receive with explicit settle, so failed handlers cause Service Bus to redeliver until `MaxDeliveryCount` is hit and the message is moved to the queue's dead-letter sub-queue.

Server A (Low Side)		Azure Service Bus Namespace		Server B (High Side)
+-----+		+-----+		+-----+
		Queue: (pre-created)		ReceiveMessages
Send message	--->	PeekLock receive	--->	(batch up to 32,
(AMQP over TLS)		DeliveryCount + DLQ		5s max wait)
				Complete on success
				Abandon on error
+-----+		+-----+		+-----+
		(independent credentials)		
+-----+		+-----+		+-----+
Upload file blob	--->	Blob sidecar container	--->	Poll blobs (15s)
+-----+		+-----+		+-----+

13. State Management and Persistence

Cross Guard stores all persistent state in Mattermost's built-in plugin KV store. No external database is required. All state is key-value pairs accessed via the Plugin API.

KV Key Schema

Key Pattern	Value	Purpose
<code>crossguard-teaminit-{teamID}</code>	<code>[]TeamConnection</code> (JSON array)	Which connections are linked to this team.
<code>crossguard-channelinit-{channelID}</code>	<code>[]TeamConnection</code> (JSON array)	Which connections are linked to this channel.
<code>crossguard-initialized-teams</code>	<code>[]string</code> (team IDs)	Global list of initialized teams for status reporting.
<code>crossguard-connprompt-{teamID}-{connName}</code>	<code>ConnectionPrompt</code> (JSON)	Team-level inbound acceptance prompt state (pending or blocked).
<code>crossguard-chanprompt-</code>	<code>ConnectionPrompt</code> (JSON)	Channel-level inbound acceptance prompt state.

Key Pattern	Value	Purpose
<code>{channelID}- {connName}</code>		
<code>crossguard-connreq- {teamID}-{connName}</code>	<code>ConnectionRequest</code> (JSON)	Pending team-admin link request awaiting System Admin approval (request-mode flow).
<code>crossguard- chanconnreq- {channelID}- {connName}</code>	<code>ChannelConnection Request</code> (JSON)	Pending channel-admin link request awaiting Team Admin approval (request-mode flow).
<code>crossguard-rwi- {connName}- {remoteTeamName}</code>	<code>string</code> (local team ID)	Team-name rewrite reverse index.
<code>crossguard-actinb- {connName}</code>	Lease record (node ID + expiry)	Single-active-receiver KV TTL lease (45 s TTL, 15 s renewal). Holder is the cluster node currently subscribed for this inbound connection.
<code>crossguard-seqcur- {connName}- {channelID}</code>	<code>SeqCursor</code> (Epoch + last admitted Sequence)	Inbound sequencer checkpoint, persisted every 5 s so leadership handoff resumes from the previous holder's position.

There is no post-ID mapping store and no per-post deleting-flag store; both mechanisms belonged to the pre-shared-channels lifecycle and were removed when the framework took ownership of the post-state lifecycle. The framework's own state (sync cursors, remote-user records) lives in Mattermost's main database, outside the plugin KV namespace.

Caching Layer

An optional caching layer wraps the base KV store to reduce API calls for frequently accessed data:

Cache	Size	TTL	Invalidation Event
Team connections	64 entries	15 minutes	<code>cache_inv_teaminit</code>
Channel connections	1024 entries	15 minutes	<code>cache_inv_chaninit</code>

Cache	Size	TTL	Invalidation Event
Initialized teams list	1 entry	15 minutes	<code>cache_inv_initteams</code>
Rewrite index	256 entries	15 minutes	<code>cache_inv_rwindex</code>

The cache is cluster-aware: write operations publish invalidation events to all cluster nodes via `PluginClusterEvent` . This ensures cache consistency in multi-node Mattermost deployments.

CAS Operations

List modifications (adding or removing a team connection, channel connection, or initialized team) use atomic compare-and-set (CAS) with 5 retries. This prevents race conditions when multiple admin actions occur simultaneously. All modifications have all-or-nothing semantics (no partial updates).

14. Concurrency Model

Cross Guard's concurrency story is deliberately small: the framework owns outbound dispatch concurrency, the single-active-receiver election owns inbound concurrency across the cluster, and the inbound sequencer owns admission ordering. No plugin-level relay semaphore exists; there is no buffered channel of relay slots and no drop-when-full policy.

Single-Active Inbound Receiver

For each inbound connection, exactly one cluster node holds the active subscription at any given moment. An elector goroutine (`server/inbound_election.go`) per inbound connection takes and renews a KV TTL lease (`crossguard-actinb-{connName}`) with a 45-second TTL and a 15-second renewal interval (a 3x safety ratio). Only the lease-holder calls `QueueProvider.Subscribe` and `WatchFiles` . Other nodes idle, observe the lease's `ExpiresAt` field, and race to take it when it ages out. The lease is released cleanly on graceful shutdown and ages out on crash. This design makes duplicate inbound processing in a multi-node cluster structurally impossible during steady state; the sequencer described next handles the narrow handoff window.

Inbound Sequencer

`inboundSequencer.Admit` in `server/inbound_sequencer.go` is the single admission gate for inbound envelopes. It keys on `(Epoch, ConnName, ChannelId, Sequence)` and enforces three invariants:

- **Deduplication.** Any envelope at or below the per-channel cursor is dropped. Absorbs at-least-once redeliveries and any envelope a previous lease holder may have processed before handoff.
- **Bounded reorder.** Envelopes ahead of the cursor sit in a small in-memory buffer; gap-fill envelopes promote the cursor and release buffered successors in order.
- **Audited gap-deadline.** A 1-second ticker drains the sequencer of envelopes whose missing predecessors never arrived. Lost ranges are audited via the `InboundSeq*` numeric error codes.

A new `Epoch` resets the per-channel cursor (the sender process generation has changed). A 5-second checkpoint ticker persists cursors to `crossguard-seqcur-{connName}-{channelID}` so leadership handoff resumes from the previous holder's position rather than from envelope zero.

Outbound Concurrency

On the outbound side, the shared-channels framework drives dispatch concurrency. The framework invokes `onSharedChannelsSyncMsg` serially per `(remote, channel)` and waits for the return before advancing the framework cursor. The plugin's hook is therefore reentrant by hook, not gated by a plugin-side semaphore. Envelope marshalling, Epoch / Sequence stamping, and the `QueueProvider.Publish` call all complete inside the hook; framework-level retry on hook-return-error is the only retry layer.

Connection-Pool Protection

The outbound and inbound connection maps and the live plugin configuration each sit behind a `sync.RWMutex` :

Resource	Lock Type	Read Lock (Concurrent)	Write Lock (Exclusive)
Outbound connections	<code>sync.RWMutex</code>	Publishing envelopes	Reconnection on config change
Inbound connections	<code>sync.RWMutex</code>	Subscription access	Reconnection on config change, lease handoff

Resource	Lock Type	Read Lock (Concurrent)	Write Lock (Exclusive)
Configuration	<code>sync.RWMutex</code>	Reading current config	Config update

Goroutine Lifecycle

- Background goroutines (elector renewers, checkpoint ticker, gap-deadline ticker, transport readers) are tracked via a `sync.WaitGroup`.
- Context cancellation signals shutdown to all goroutines.
- `OnDeactivate` sequence: cancel context, release inbound leases, close transport subscriptions, wait for the `WaitGroup`, close outbound providers. The framework hooks are not reentrant during shutdown; the framework will not call into a deactivating plugin.

15. Operational Monitoring and Troubleshooting

Numeric Error Codes

Every operator-facing log call (`LogInfo`, `LogWarn`, `LogError`) in non-test code carries a unique numeric `error_code` as its first key-value pair, sourced from `server/errcode/codes.go`. The integer is the stable contract for log grep across plugin versions; the identifier (constant name) may be renamed but the number never changes once assigned. Codes are allocated by source file in 1000-wide blocks: `hooks.go` uses 10000-10999, `inbound.go` uses 15000-15999, the inbound election uses 15200-range, the inbound sequencer uses 15300-range, and so on. The complete list with descriptions and troubleshooting steps lives in [the Error Codes page](#), regenerated from `server/errcode/codes.go` by `make generate-error-codes`.

Operator-relevant code families include:

- **`OutboundSyncMsg*` / `OutboundAttachment*` / `OutboundProfileImage*` / `Ping*` (10100-range):** outbound hook events: provider mismatches, channel lookup failures, publish failures, file fetch failures, ping unhealthy.
- **`InboundActive*` (15200-range):** single-active election events: lease acquired / renewed / released, leader handoff detected, lease lost.

- **InboundSeq* (15300-range)**: sequencer events: gaps closed, gaps audited as lost, epoch reset, cursor checkpoint persisted. These are the compliance-relevant audit codes for envelope ordering.
- **AzureSPAuditConstructed (12010)**: Azure Service Principal credential construction audit (logs tenant ID, client ID, Azure cloud, and a short hash of the secret).
- **ServiceBusSPCredentialFailed (26007)**: Service Bus SP credential construction failure (sanitized error).

Status Commands

- `/crossguard status` : shows team and channel linkage state. System Admins see a global overview across all teams and connections. Team members see their team / channel status.

REST API Status Endpoints

Endpoint	Permission	Returns
<code>GET /api/v1/status</code>	System Admin	Global status: all teams, connections, lease holders, and any orphaned KV state.
<code>GET /api/v1/teams/{id}/status</code>	Team member	Team-specific status, including pending or blocked prompts.
<code>GET /api/v1/channels/{id}/status</code>	Channel member	Channel-specific status.
<code>GET /api/v1/channels/connectionstatus?ids=...</code>	Channel member	Bulk channel-connection lookup, used by the webapp sidebar.

Connection Testing

`POST /api/v1/test-connection` (System Admin only) tests transport connectivity without affecting running connections. The handler dispatches by provider: NATS performs a connect-publish-flush cycle, Azure Queue performs a queue probe, Azure Blob performs a container probe, and Azure Service Bus performs an AMQP connect. Each test envelope carries the current `Epoch` and a generated `TestID` the receiver echoes to audit logs.

Remote-User Audit

Users synced from a remote can be enumerated by querying for users with a non-empty `RemoteId` . Each synced user carries `UserProps.RemoteUsername` with the source-side username and (optionally) `UserProps.OriginalRemoteId` indicating the upstream remote when content has passed through intermediate relays. There is no `Position` -based audit (the legacy `crossguard-sync` convention was removed).

16. Comparison with Alternative Approaches

Capability	Cross Guard (NATS / Azure)	Shared Database	Direct API	Email Bridge	Webhook Relay
Server independence	Full	None (shared DB)	Partial	Full	Full
Network isolation	Yes (NATS in DMZ / Azure private endpoint)	No (DB access)	No (HTTP access)	Yes	No (HTTP access)
Real-time messaging	Yes (sub-ms NATS / 5-15s Azure)	N/A	Yes	No (minutes)	Yes
Edit/delete sync	Yes	N/A	Possible	No	Manual
Reaction sync	Yes	N/A	Possible	No	No
Thread support	Yes (framework RootId resolution)	N/A	Possible	No	No
Admin opt-in control	Team + channel level	N/A	N/A	N/A	Per-webhook
Transport security	TLS 1.2+, mTLS, ACLs (NATS); TLS always, RBAC or shared key (Azure)	DB TLS	HTTPS	SMTP TLS	HTTPS

Capability	Cross Guard (NATS / Azure)	Shared Database	Direct API	Email Bridge	Webhook Relay
Message authentication	NATS token / credentials / mTLS; Azure shared key, SAS, or Service Principal	DB auth	API auth	DKIM/SPF	Shared secret
Resilience	Auto-reconnect	DB failover	Retry logic	Store-and-forward	Manual
Data minimization	Text + optional files (disabled by default, per-connection toggle, type filtering). Azure blobs deleted after processing.	Full DB access	Full API access	Full email	Configurable
Audit trail	Numeric error codes in plugin logs (stable grep contract); framework-side remote-user attribution	DB audit log	API logs	Mail logs	Webhook logs

17. CDS Deployment Checklist

Hardened Configuration

The following checklist is recommended for Cross Guard deployments in classified or sensitive environments. For a detailed STRIDE threat analysis of each risk, see the [Threat Model](#).

- 1 Enable `RestrictToSystemAdmins`** : Lock all connection management commands to System Admins. Prevents Team Admins and Channel Admins from enabling relay without organizational approval. If you need to delegate linking to non-sysadmins without granting them unsupervised authority, enable `AllowTeamAdminRequests` and / or `AllowChannelAdminRequests` so the corresponding admin tier can submit link requests that a higher tier approves via interactive DM (see Section 9, Request-Mode Workflows).
- 2 Prefer Azure Service Principal authentication:** For Azure Queue, Azure Blob, and Azure Service Bus, configure `auth_mode: "service-principal"` with `tenant_id`, `client_id`, and

`client_secret` (and `service_bus_namespace` for Service Bus). Scope each Service Principal to data-plane-only RBAC on the specific queue, container, or namespace; rotate the secret in Azure AD without touching plugin config. Pre-provision queues and containers (auto-create is skipped in SP mode).

- 3 **Inject connections JSON via environment variables:** Source the `MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` and `...INBOUNDCONNECTIONS` variables from a secret store (Kubernetes Secret, systemd `LoadCredential`, Vault, Key Vault). Keeps shared keys, SAS connection strings, NATS tokens, and SP `client_secret` values out of `config.json` on disk and out of the Mattermost configuration database (see Section 11, Secret Storage and Env-Var Injection).
- 4 **Enable TLS with mutual certificate authentication (NATS):** For NATS connections, configure `tls_enabled`, `client_cert`, `client_key`, and `ca_cert`. For Azure connections, TLS is always enabled by the Azure endpoint.
- 5 **Deploy a dedicated transport endpoint:** For NATS, do not share the NATS server with other applications. For Azure, use a dedicated Storage account and a dedicated Service Bus namespace for Cross Guard traffic. Reduces the risk of message content exposure ([T-INFO-1](#)).
- 6 **Configure transport-level access control:** For NATS, restrict publish and subscribe permissions per subject per client via NATS ACLs. For Azure, use Azure RBAC scoped to the queue / container / namespace. Avoid the namespace-wide `RootManageSharedAccessKey` Service Bus rule in production.
- 7 **Use separate subjects, queues, or namespaces per relay direction:** Configure independent transport resources for each direction (e.g. NATS `crossguard.a-to-b` versus `crossguard.b-to-a`; separate Azure queues or Service Bus namespaces). Prevents transport-level echo and isolates the blast radius of a credential compromise.
- 8 **Place the transport endpoint in a controlled network segment:** For NATS, deploy the server in a DMZ, enclave, or guard network between the two domains. For Azure, use private endpoints or VNet service endpoints to restrict storage account and namespace access to authorized networks.
- 9 **Monitor the operator-facing audit codes:** Watch for `InboundActive*` (15200-range; election anomalies), `InboundSeq*` (15300-range; lost ranges, epoch resets, replay detection), and the

Service Principal credential audit (`AzureSPAuditConstructed` , code 12010). The complete numeric error code list is at error-codes.html.

- 10 **Validate emitted envelopes against the XSD:** Run `make docker-integration-test-validate-wire` as part of the release pipeline. The mode archives every envelope the running plugin emits and validates each against `schema/crossguard.xsd` , catching any regression that would put non-conformant content on the wire.
- 11 **Audit synced remote users periodically:** Enumerate users with a non-empty `RemoteId` to verify the population of synced accounts matches the expected remote topology. Each carries `UserProps.RemoteUsername` showing the source-side username.
- 12 **Enforce `crossguard.` subject prefix (NATS):** All NATS subjects must start with `crossguard.` (enforced at configuration time). Prevents accidental subscription to unrelated NATS traffic.
- 13 **Document and review team name rewrite rules:** Maintain an inventory of all rewrite mappings. Rewrites change message routing and should be reviewed as part of the security configuration.
- 14 **Test connections regularly:** Use the admin console test button or the `POST /api/v1/test-connection` endpoint to verify transport connectivity without affecting running connections.
- 15 **Review the Threat Model:** The [Threat Model page](#) provides a detailed STRIDE analysis of identified threats with severity ratings, code paths, mitigations, and residual risks.

18. References

Overview

Getting started, key concepts, permission model, and federation overview.

Slash Command Reference

All 9 slash commands with arguments, permissions, and examples.

Admin Configuration Guide

Transport connection setup, relay settings, and example configurations.

REST API Reference

All REST endpoints with request/response examples and authentication details.

Threat Model

STRIDE analysis, trust zones, 14 identified threats, and CDS deployment recommendations.